Tessellate AI — Architecture & Usage Guide

Audience: anyone joining this repo who needs to understand what the system does, how the pieces fit together, and where the honest limits are. **Basis:** this document was written by reading the source in `core/`, `api/`, `ui/`, `data/` and `models/`, and by running the shipped model against real requests. Numbers marked **verified** were measured on this machine (Python 3.11.6, CatBoost 1.2.10) on 2026-11-01, after the rejoin-traffic features landed. **Related files:** README.md (marketing summary) · this file (how it actually works).

---

1. What the project is

Tessellate AI answers one question a Formula 1 pit wall asks dozens of times per race:

> "*This car, right now, on these tyres — should we box (pit) this lap, or stay out?*"

It answers it by predicting the driver's **finishing position** twice, for two parallel universes of the same race moment, and recommending whichever universe finishes in a lower (better) number. That "score the same moment twice, changing only one decision flag" pattern is what makes it a **counterfactual** evaluator rather than a plain predictor.

The name comes from tessellation: tiling a plane with shapes and no gaps. A race is tiled into stints (tyre periods), and the model fits those tiles into the remaining race distance.

What it is not: it is an independent, unofficial portfolio project. It is not affiliated with F1, the FIA, or any team, and it is not a race-legal decision tool — see §11 Limitations.

---

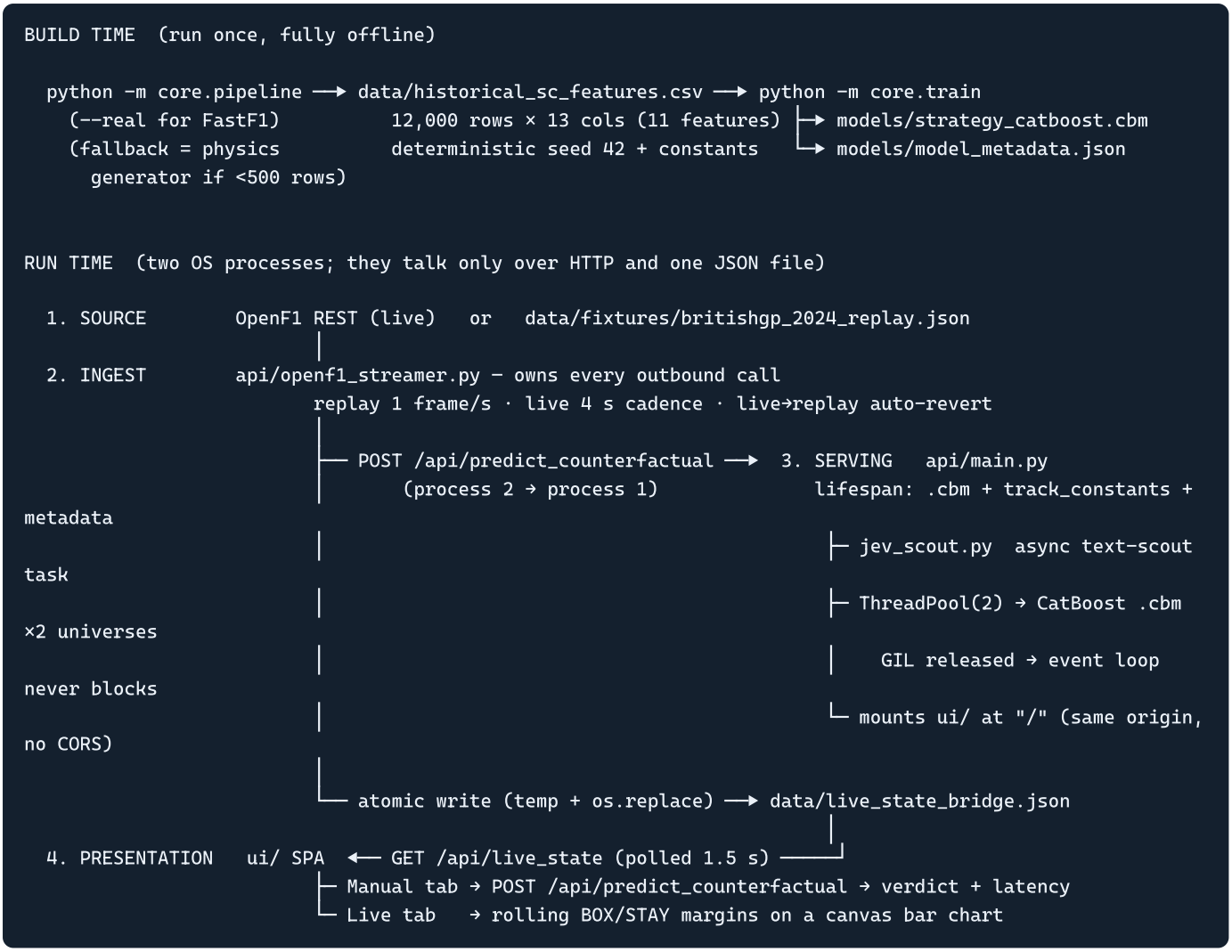
2. The idea in one paragraph (for readers new to ML)

A model is trained on lots of historical race moments. Each moment is a row of numbers — which circuit, which lap, what position, how old the tyres are, how big the gaps are, what the pit lane costs and where a stop would rejoin you — plus one label: *where that car actually finished*. Once trained, you can feed it a moment it has never seen and it estimates the finishing position. The trick that makes this project interesting is the column `will_pit`: for every snapshot you ask twice, once with `will_pit = 1` ("this car pits") and once with `will_pit = 0` ("this car stays"). Subtract the two predictions and you get the **positional value of the decision** — the thing the pit wall actually cares about.

---

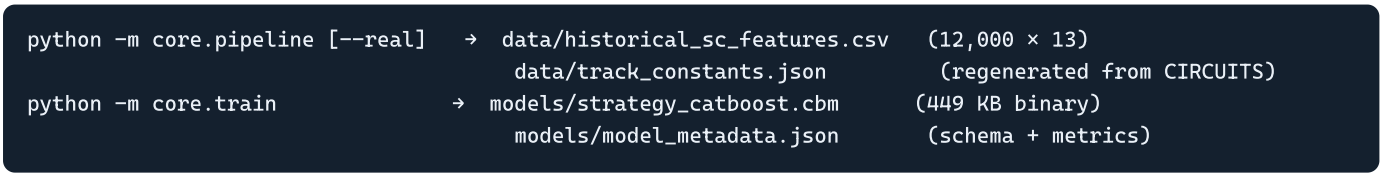
3. System architecture

The system is **two OS processes plus a browser** at run time, and a **two-command build pipeline** that produces the artifacts they load. The process split is the load-bearing design decision: network I/O can never freeze inference.



If Process 2 dies, nothing about inference changes: the API, the model and the dashboard all keep serving, the Live tab simply reads `{"available": false}` and the UI holds its last good frame. Swapping the ingest source — replay file, live OpenF1, or a future MQTT feed — touches no serving code.

Offline design/build flow (what creates the artifacts)



---

4. Module reference

Module	Responsibility	Key symbols (file:line)
`core/config.py`	Single immutable path anchor. `PROJECT_ROOT = Path(__file__).resolve().parent.parent` (`core/config.py:6`) then derives `DATA_DIR`, `MODELS_DIR`, `FIXTURES_DIR`, `FRONTEND_DIR`, `BRIDGE_PATH`, and `os.makedirs` them on import (`:13–16`). Every other module imports from here, so no module ever depends on the current working directory.	`PROJECT_ROOT`
`core/pipeline.py`	Builds the feature matrix and owns the one canonical schema. Two interchangeable producers with an identical column contract: `extract_race_grade_features()` (`core/pipeline.py:138`, FastF1, seasons 2023–2024) and `generate_realistic_features()` (`:213`, offline physics simulator). `OUTPUT_COLUMNS` (`:19`) → `MODEL_FEATURES` (`:37`) is the single source both trainer and server import. `_write()` (`:309`) emits `data/historical_sc_features.csv`; `write_track_constants()` (`:316`) re-exports `data/track_constants.json` from the same `CIRCUITS` dict (`:61–82`); `__main__` (`:331`) implements the fallback threshold.	`CIRCUITS`, `COMPOUND_DEGRADATION`, `COMPOUND_CLIFF_LAP`, `MODEL_FEATURES`, `FALLBACK_THRESHOLD = 500`
`core/train.py`	Trains and evaluates. Imports `MODEL_FEATURES as FEATURES` (`core/train.py:13`) rather than restating it; `CAT_FEATURES` (`:21`), hand-rolled seeded `_group_kfold_indices()` (`:27`, no scikit-learn dependency), `_metrics()` (`:44`), `train_with_grouped_cv()` (`:52`) → CV loop, full-dataset refit, native `.cbm` save (`:97`), metadata dump (`:105`), quarantined ONNX attempt (`:129`).	`CV_PARAMS`, `FINAL_PARAMS` (`:23–24`)

Module	Responsibility	Key symbols (file:line)
`api/main.py`	The serving tier. Imports the same canonical list (`api/main.py:15`). `bootstrap_context` lifespan (`:25–44`) hard-fails at boot if the `.cbm` is missing, otherwise loads model + `track_constants.json` + metadata once. `InferenceRequest` (`:51–62`) is the Pydantic guard rail. `compute_model_score()` (`:65`) is the synchronous worker that runs off-loop. `_build_feature_row()` (`:71–89`) is the counterfactual contract. `predict_counterfactual()` (`:92–146`) is the hot path; `rejoin_context` is echoed at `:139`. `live_state()` (`:168`) reads the bridge defensively. `StaticFiles` mount is last (`:187`) so API routes win and the SPA is same-origin (no CORS).	`executor = ThreadPoolExecutor(max_workers=2)` (`:19`)
`api/jev_scout.py`	Unstructured-text layer. `_tire_urgency()` (`api/jev_scout.py:10`) maps radio keywords to Critical/Elevated/Low; `analyze_text()` returns an SC escalation risk float plus that label. Deliberately shaped like a future LLM call: `async`, isolated, keyword tiers inside — so a real model swaps in behind the same signature.	`TIRE_KEYWORDS_CRITICAL`, `SC_KEYWORDS`
`api/openf1_streamer.py`	Standalone ingest process. `seed_replay_fixture()` (`api/openf1_streamer.py:24–87`) auto-generates the 60-frame Silverstone 2024 replay if absent, deriving each car's gaps from its own wear; `write_bridge()` (`:89`) does temp-file + `os.replace`; `post_snapshot()` (`:96`) calls the API and now forwards `gap_ahead/gap_behind`; `resolve_live_session()` (`:117`) and `poll_live_packet()` (`:143`) implement real OpenF1 polling; `run_background_poller()` (`:176`) is the loop with the live→replay auto-revert and the rolling record (`:213–221`).	`FOCUS_DRIVERS = (4, 44, 16)`
`ui/`	Zero-dependency SPA, no build step, no Node. `index.html` (162 l) two tabs + metrics strip, rejoin read-out at `:107–110`; `app.js` (204 l) `loadHealth` (`:17`), `loadTracks` (`:36`), evaluate handler (`:51`) with the rejoin block (`:92–100`), `pollLive` (`:117`), hand-drawn `drawChart` canvas bar chart (`:154`), decision table (`:183`); `styles.css` (164 l). Polling cadence: live 1500 ms, health 10000 ms, clock 1000 ms (`:202–204`).	`signed()` (`app.js:4`)

Module	Responsibility	Key symbols (file:line)
`data/`	`track_constants.json` (serving circuit memory), `historical_sc_features.csv` (generated matrix), `live_state_bridge.json` (generated, gitignored), `fixtures/britishgp_2024_replay.json` (generated on first run), `fastf1_cache/` (FastF1 HTTP cache, gitignored).	—
`models/`	`strategy_catboost.cbm` = the production C++ inference binary (gitignored, regenerable); `model_metadata.json` = committed schema + metrics + versions, which is how the server reports its own provenance.	—

5. The feature contract

The model consumes exactly **eleven** columns. There is now one definition of that list: `core/pipeline.py:19` `OUTPUT_COLUMNS` (the full label matrix, 13 columns) derives `MODEL_FEATURES` at `:37` by dropping `compound_degradation` and `final_position`, and both `core/train.py:13` and `api/main.py:15` import it. Renaming or reordering a column in the generator therefore propagates to training and serving automatically instead of silently diverging.

Feature	Type	Meaning	Notes
`track_id`	categorical	circuit name	Native CatBoost categorical. Its importance has fallen to **0.84%** (was 3.3%) because circuit identity now enters numerically through `pit_loss_s` — which is what lets an unseen track generalise.
`laps_remaining`	int	laps left in the race	Bounds how much a tyre advantage can be cashed in.
`current_position`	int 1–20	running order now	**Dominant** feature (45.7% importance, down from 66.2% as the new columns earned share).
`tire_age`	int laps	stint length so far	Degradation driver; interacts with the compound's cliff lap.
`tire_compound`	categorical	SOFT / MEDIUM / HARD	Native categorical. 4.8% importance (was 1.6%) once each compound got its own cliff.
`gap_ahead`	float s	delta to the car in front	Clean air vs traffic. Alone it is weak (0.68%) — it matters as an *offset against pit loss*.
`gap_behind`	float s	delta to the car behind	Defensive pressure: 1.77%. How close the undercut threat is.

Feature	Type	Meaning	Notes
`pit_loss_s`	float s	the circuit's stationary pit-lane delta (17.0–24.5)	**New.** The cost side of a stop, taken from `track_constants.json` at serve time. 1.86%.
`rejoin_gap_s`	float s	`gap_ahead − pit_loss_s`	**New.** Where the stop leaves you: negative = you rejoin behind the car you were attacking, positive = the undercut already covers the loss and you land in clean air. 1.07%.
`driver_pace_index`	float s	driver's median quick-lap pace	Race-long skill proxy. 7.4%.
`will_pit`	int 0/1	the decision flag	The only thing that changes between the two universes. 22.5% (was 11.5%) — giving the model the rejoin geometry roughly doubled how much the decision itself explains.

Target: `final_position`. One extra column, `compound_degradation`, is written into the CSV by the pipeline (`core/pipeline.py:40`, per-compound seconds-per-lap wear: SOFT 0.085, MEDIUM 0.050, HARD 0.032) but is deliberately excluded from `MODEL_FEATURES`: the compound categorical already carries it, and a second encoding of the same quantity would only split the importance. It is generator physics and documentation of intent.

Why `rejoin_gap_s` is a column rather than a rule. A counterfactual server must not add pit-loss seconds to one universe by hand — §7 explains the double-counting that causes. The clean way to get "traffic after the stop" into a tree model is to hand it the *state* the stop produces (the deficit you rejoin into) and let `will_pit × rejoin_gap_s` be learned. Both columns are identical on both sides of the comparison, so the contract stays "flip one flag".

---

6. How the training and evaluation work

Model. `CatBoostRegressor` (gradient-boosted decision trees) with `loss_function="RMSE"`. Two hyper-parameter sets on purpose (`core/train.py:23–24`): `CV_PARAMS` = 250 iterations for honest fold scoring, `FINAL_PARAMS` = 400 iterations for the production refit on all data. Depth 6, learning rate 0.05, `random_seed=42` throughout so reruns are reproducible.

Why CatBoost. It handles categorical strings (`track_id`, `tire_compound`) natively without one-hot explosion, and it exports to a compiled C++ binary (`.cbm`) that loads and predicts without a Python ML stack — which is why the serving tier is fast and dependency-light. Native `.cbm` serving also sidesteps ONNX's categorical-export gap, which is what the quarantined export attempt at `core/train.py:129` warns about.

Validation — GroupKFold(3) by circuit (`core/train.py:27–42`, loop at `:71–84`). Rows are grouped by `track_id`, the unique circuit names are shuffled with a seeded RNG, and each of the three folds trains on two thirds of the circuits and scores on the third it has never seen. The point: random row splits would leak the same circuit's noise into both sides and flatter the metric. Grouped-by-circuit means the reported numbers describe **generalisation to an unknown track**, which is exactly the deployment situation. Each fold prints which circuits it held out, so the claim is inspectable. This scheme is doing more work than before: the two new pit columns are circuit *summaries*, so a fold can score a circuit it never trained on and still know what its pit lane costs.

Baseline. Every fold also scores the naive prediction "the car finishes where it is running now" (`df["current_position"]`). A model that beats a leaderboard refresh is the minimum bar for this being useful at all.

Reported metrics (`models/model_metadata.json`, and **verified** against it):

Measure	Value	Reading
CV RMSE	**1.144** positions	penalty on the odd big miss
CV MAE	**0.856** positions	typical absolute error
Naive baseline MAE	**3.322** positions	do-nothing comparison
Improvement	**≈ 74%** (1 − 0.856/3.322 = 0.742)	was 40% before the rejoin columns
Training footprint	12,000 rows · 20 circuits · 13 columns (11 features)	**verified** from the CSV

Read that improvement honestly. Most of the step from 1.480 → 0.856 positions is not a smarter model; it is a generator that now writes labels from mechanisms the feature row actually contains. Before, `pit_loss` and the rejoin deficit existed only inside the simulator's arithmetic, so part of every label was structurally unexplainable and the residual error was inflated by construction. Adding the columns removed that blind spot. The metric is still measured on unseen circuits with a seeded split, so it is a valid statement *about this data* — and still **not** a claim of race-grade accuracy (§11 item 1).

Feature importance (from the production model): `current_position` 45.7, `will_pit` 22.5, `tire_age` 9.2, `driver_pace_index` 7.4, `tire_compound` 4.8, `laps_remaining` 4.3, `pit_loss_s` 1.9, `gap_behind` 1.8, `rejoin_gap_s` 1.1, `track_id` 0.8, `gap_ahead` 0.7. Honest reading: position dominance is expected — where you are running is the strongest predictor of where you finish — and it is also why the *decision value* must be read from the **difference between the two projections** (`expected_position_gain`), not from importances. What changed is where the traffic signal lives: `gap_ahead` alone stayed near-useless (0.14% → 0.68%) because a gap means nothing until you set it against the pit loss, while `will_pit` doubled and `tire_compound` tripled once the labels carried a real rejoin cost and a per-compound cliff.

---

7. The counterfactual contract (the core mechanism)

`api/main.py:71–89` `_build_feature_row()` is the whole idea in nineteen lines. For a request it builds the identical observed snapshot — position, tyre age, compound, gaps, pace, **pit-lane delta and rejoin gap** — and sets only `will_pit` to 1 or 0. The server does **not** zero the tyre age for the "box" universe and does **not** add pit-loss seconds or queue places manually.

```
# api/main.py:71–89 (abridged)
def _build_feature_row(request, will_pit, pit_loss_s) -> dict:
    return { ...,
             "pit_loss_s": pit_loss_s,                       # circuit state, same both sides
             "rejoin_gap_s": round(request.gap_ahead - pit_loss_s, 2),  # same both sides
             "will_pit": will_pit }                          # the only difference
```

Verified: the two rows differ in exactly one key. Sending the same request through the builder twice and diffing the dicts returns `{'will_pit': (1, 0)}` and nothing else. That is the contract, checked rather than asserted.

Why this shape: during labelling (`core/pipeline.py:169–195` real, `:240` + `:277` synthetic) `will_pit` is correlated with what actually happened to those cars, so the model has already internalised "pitting at Monaco costs places, pitting at Silverstone on worn Softs gains them" *as a function of the other features*. Re-adding pit loss at inference time would

double-count it and make the model's circuit memory useless. Note the distinction that makes traffic-after-the-stop representable at all: **the rejoin deficit is state, the pit stop is the decision.** `rejoin_gap_s` is therefore allowed — indeed required — to be a feature, because the same value has different consequences depending on which universe you are in. That interaction is exactly what `will_pit` doubles its importance on.

Verified decision behaviour — the shipped `.cbm`, position 6, 20 laps remaining, SOFTs, 1.0 s gaps, 90.0 s pace, scored both ways:

Circuit	Tyre age	BOX predicts	STAY predicts	Verdict	Gain
Monaco Grand Prix	4	14.33	6.16	**STAY OUT**	−8.17
Monaco Grand Prix	20	13.41	7.67	**STAY OUT**	−5.74
Monaco Grand Prix	28	11.47	10.43	**STAY OUT**	−1.04
British Grand Prix	4	10.28	6.46	STAY OUT	−3.82
British Grand Prix	16	9.59	7.50	STAY OUT	−2.09
British Grand Prix	20	9.14	8.18	STAY OUT	−0.96
British Grand Prix	24	8.29	9.45	**BOX**	+1.16
British Grand Prix	28	7.01	11.28	**BOX**	+4.27

Monaco stays "STAY OUT" at every tyre age probed — a 24.5 s pit lane into a 6-place queue that the remaining race distance (window `20/78`) cannot repay. The British call flips between 20 and 24 laps of Soft wear, i.e. right after the Soft cliff lap (`COMPOUND_CLIFF_LAP["SOFT"] = 15` is where wear starts compounding), and the advantage then *grows* with age: +1.16 → +4.27. The same 20-lap-old Soft produces opposite advice at two circuits.

Traffic after the stop — the part a per-circuit average cannot express. Same car, same 22 laps of Soft wear, same 20 laps remaining, only the gap ahead changed:

Gap ahead	`rejoin_gap_s`	BOX	STAY	Verdict	Gain
0.8 s	−20.70	8.79	8.74	STAY OUT	−0.05
5.0 s	−16.50	8.43	8.32	STAY OUT	−0.12
10.0 s	−11.50	7.99	8.09	**BOX**	+0.10
22.0 s	+0.50	6.90	7.59	**BOX**	+0.70
40.0 s	+18.50	6.67	7.49	**BOX**	+0.81

The BOX projection improves monotonically as the car ahead escapes, because the stop stops costing a place; once `rejoin_gap_s` goes positive the model has seen the rejoin land in clean air and the fresh tyres are pure profit. Gap *behind* moves the other way — 0.8 s behind → −0.05, 30 s behind → −0.85 — which is the undercut threat: a car within DRS range is a reason to cover him, a car 30 s back is not. Both directions are the behaviour a strategy engineer checks for; neither was expressible when the only traffic signal was a circuit-level average.

And on identical inputs, the circuit still decides:

Circuit (P6 · 20 laps · age 20 SOFT · 1 s / 1 s)	`pit_loss_s`	Gain	Verdict
Monaco Grand Prix	24.5	−5.74	STAY OUT
Singapore Grand Prix	24.0	−3.22	STAY OUT

Circuit (P6 · 20 laps · age 20 SOFT · 1 s / 1 s)	`pit_loss_s`	Gain	Verdict
Dutch Grand Prix	18.5	−2.14	STAY OUT
Italian Grand Prix	21.5	−1.55	STAY OUT
British Grand Prix	21.5	−0.96	STAY OUT

Dutch has the *shortest* pit lane and the *worse* margin: Zandvoort's traffic index is 4 versus Italy's 3, so cheap lane ≠ cheap stop. The model is reading both cost channels, not just distance.

Sign convention in the response: `expected_position_gain = STAY − BOX`. Positive means boxing is better. The verdict is `"BOX" if finish_box < finish_stay else "STAY OUT"` (`api/main.py:127`) because in motorsport **lower is better**.

---

8. Hot path: one request, step by step

1. **Receive + validate.** FastAPI hands the JSON body to `InferenceRequest` (`api/main.py:51–62`). Pydantic rejects out-of-range values before any compute: `current_position` 1–20, `tire_age` 0–60, `laps_remaining` 0–78, `gap_ahead` / `_behind` 0–300, `driver_pace_index` 60–120. Bad input returns HTTP 422 and costs no CPU.
2. **Circuit memory lookup.** `TRACK_MEMORY.get(request.track_name)` (`:99–101`) reads that circuit's **`pit_loss`** — now a live model input, not a decoration — and sets `track_fallback_used`. An unknown circuit name falls to `DEFAULT_PIT_LOSS_S = 21.0` and flags itself in the response rather than failing.
3. **Fan out.** `asyncio.create_task(analyze_text(...))` starts the non-blocking text scout (`:104–106`) and *does not await it*. Two feature rows are built (`:108–109`), identical except `will_pit`.
4. **Offload the CPU.** Both predictions go through `loop.run_in_executor(executor, compute_model_score, ...)` (`:114–115`) on the 2-thread pool, then `await asyncio.gather(...)` (`:117`). CatBoost's predict is C++ and releases the GIL, so the two universes genuinely run in parallel while the event loop keeps serving other requests — including dashboard polling. This is the answer to the classic FastAPI failure mode of a synchronous model call freezing the whole server.
5. **Rejoin.** `jev_indicators = await jev_task` (`:126`) — the ~40 ms text-scout round-trip was paid for while the trees ran.
6. **Decide + return.** Compare the two projections, compute the gain, echo the `rejoin_context` the model was shown (`:139–143`), report `inference_ms` measured with `time.perf_counter()` around the tree work only (`:111`, `:124`).
7. **Failure path.** Any exception inside the executor cancels the pending text task and raises a structured HTTP 500 (`:118–123`) rather than leaving a dangling coroutine.

Latency, verified. Warm single-row in-process predict: **0.93 ms** averaged over 200 calls (it was 0.44 ms at nine features — two more columns and a 449 KB binary cost roughly half a millisecond). The HTTP path as recorded in `data/live_state_bridge.json` over a 40-frame replay run: **min 3.5 ms, median 8.5 ms, max 245 ms**, the max being the cold first call (thread-pool spin-up and first-predict warm-up). The README's "~5–15 ms core inference, target <200 ms" still describes the warm HTTP path, and the cold spike is what that <200 ms budget is actually for.

---

9. Ingest, bridge and live feed

`api/openf1_streamer.py` runs as its own process and is the only component that touches the network for race data.

Replay path (default). `seed_replay_fixture()` (`api/openf1_streamer.py:24–87`) writes 60 frames — laps 21–40 × 3 cars (Norris #4, Hamilton #44, Leclerc #16) — for the British GP 2024. **Verified**: 60 frames, laps 21→40, drivers {4, 16, 44}. The fixture is not random. Each car runs one long stint on a compound fixed at its own stop lap (`stints = {4: (14, SOFT), 44: (8, MEDIUM), 16: (12, HARD)}`, `:37`); a safety car is deployed at lap 35; the radio escalates past *that compound's* cliff lap; and crucially the two gaps are derived from the car's own degradation —

```
gap_ahead  = 0.8 + COMPOUND_DEGRADATION[compound] * tire_age**2 / 2   # integral of your own loss
gap_behind = max(0.4, 9.0 - 0.5 * tire_age)                           # a chaser closing up
```

That single line is what makes the traffic-after-the-stop term observable in the replay rather than only in the manual tab: as a driver loses time to the car ahead, his accumulated deficit eventually exceeds Silverstone's 21.5 s pit lane, the rejoin lands in clean air, and the call flips. **Verified through the shipped** `.cbm`: Norris's margin runs −3.69 positions at lap 21 → −0.43 at lap 38 → **+0.47 at lap 39** → **+1.17 at lap 40**, with `rejoin_gap_s` crossing zero at lap 37 (+1.78). Hamilton on 32-lap Mediums (−1.27) and Leclerc on 28-lap Hards (−3.03) never flip, because their compounds cliff later and Leclerc in particular is still losing only 13 s to the car ahead. One lap, three cars, three different correct answers.
`race_control_text` is attached to the first focus driver only, so SC detection is exercised once per lap, not three times.

> ⚠️ Fixing the fixture also exposed a real bug: Hamilton's stint start was recorded as lap 22 while the window opens at lap 21, so frame 2 produced `tire_age = -1`, Pydantic answered 422, and `post_snapshot()` swallowed it as `{"status": "server error 422"}` — the dashboard showed "Engine verdict pending" for that frame instead of an error. The stint start is now lap 8 and the ages are legal.

Live path. `--live` → `resolve_live_session()` (`:117`) takes the latest scheduled session and applies OpenF1's own "live" definition (start−30 min → end+30 min) because the free tier does not support `live_only=true`.
`poll_live_packet()` (`:143`) pulls `/laps`, `/race_controller`, `/chat` in a single flight and sleeps 4 s to stay inside the free 3 req/s limit. If no session is live, `live_mode` flips itself back to `False` and the process silently reverts to replay. Note: live tyre age/compound are placeholder derivations from the free feed (`:161–162`) and `gap_ahead` / `gap_behind` are hard-coded 1.0/1.0 (`:166–167`) — OpenF1's free endpoints carry no time intervals, so on a real Sunday the traffic term contributes nothing until an interval feed is wired in.

Bridge protocol. Each evaluated frame becomes `{latest_snapshot, latest_decision, rolling_decisions[-40:]}` (`:222–227`) — the rolling rows now carry `rejoin` alongside `call` / `gain` / `inference_ms` (`:213–221`) so the Live table shows where each stop would have landed — and is published with **write-to-temp-then-`os.replace`** (`:89–92`). On the same filesystem `os.replace` is atomic, so the server never observes a half-written file. The read side still defends itself (`api/main.py:168–185`): a `FileNotFoundError`, `JSONDecodeError` or `OSError` — Windows can briefly lock the target — returns `{"available": false}` and the dashboard keeps its last good frame instead of blanking.

Frame cadence: replay 1 frame/s, live 1 packet/4 s, UI poll 1.5 s.

---

10. Data pipeline and the fallback guardrail

`python -m core.pipeline` decides *by itself* which producer to use (`core/pipeline.py:331–347`):

- No flag → `generate_realistic_features()` (12,000 rows, offline, deterministic `default_rng(42)`).
- `--real` → `extract_race_grade_features()` over seasons 2023–2024 with a local FastF1 cache in `data/fastf1_cache/`. Every schedule and every Grand Prix is wrapped in its own `try/except`, so one broken event skips instead of aborting the run.
- `--real` yielding fewer than `FALLBACK_THRESHOLD = 500` rows → prints the shortfall and injects the synthetic matrix anyway. The build therefore never produces a thin, unusable dataset.

The label physics (`core/pipeline.py:242–286`) is where the answer comes from, and it is deliberately written as *race mechanics*, not as a rule about the flag.

- `recovery_window = laps_remaining / total_laps` (`:243`) scales both branches: a tyre advantage is only worth what you can cash in before the flag.
- **Wear has a cliff** (`:250–251`): `wear = tire_age × compound_wear + 0.015 × max(0, tire_age − cliff_lap)²`, with `COMPOUND_CLIFF_LAP = {SOFT 15, MEDIUM 22, HARD 30}` (`:56`). This is the term that makes pitting *necessary* at some point. Without it a 30-lap Soft costs almost the same as a 20-lap one, no fresh tyre can ever repay a stop, and the engine degenerates into "always hold track position" — which is exactly what it did before this existed.
- Staying out costs `wear × window × 3.0` (`:253`), eased by up to `0.4 × clip(gap_behind − 1, 0, 6)` (`:257`) — a distant chaser buys relief, but only until nobody is within a few seconds, and an unbounded version of this term inverted the traffic signal during calibration. Boxing recovers `wear × window × 2.2` (`:258`) and pays `box_cost = traffic_index + 0.15 × (pit_loss − 20)` (`:259`). The sign here is now `+`: a longer pit lane is *more* expensive. It used to be `−`, which made Monaco's 24.5 s lane cheaper to use than a 17 s one; that was found while documenting §11 and is fixed.
- **Where you rejoin** (`:267–275`) is the new machinery. `rejoin_gap = gap_ahead − pit_loss`; if it is negative you land behind the car you were attacking, so you spend `deficit / 0.6` laps in its wake (capped at the laps actually remaining), lose `0.25 s/lap` of dirty air, and every second the car behind closes by `1.5 s`-per-place costs you more of him. Total `rejoin_traffic` is capped at 2.5 places, because a rejoin costs places, not laps — fresh rubber takes them back over a stint.
- Clean air eases the stayer too, bounded (`:283`): a 40 s gap ahead is worth no more breathing room than a 10 s one.
- `pace_skill = 90 − driver_pace_index` shifts everything for a quick car, and N(0, 0.7) noise plus a 1–20 clip (`:285–286`) keeps it plausible.

Gap coverage. A race is not always a train, so 20% of rows get the field ahead strung out to 15–45 s and 12% get a 10–35 s hole behind (`:229–236`). Without those tails `gap_ahead` never exceeds any circuit's pit loss, `rejoin_gap` is negative in every row, and the model literally cannot learn the undercut because it has never seen one — that was the first version of this section, and it is why the columns looked dead.

Non-degeneracy check, verified: across 400 snapshots drawn from the generator's own input distribution, the engine recommends **BOX in 15.2%** of them (61/400). A strategy engine that never says "pit" is as useless as one that always does, and both the always-stay and always-box failure modes appeared during this calibration before the cliff and the tails were added.

Why this matters for the rest of the doc: both producers write the identical 13-column contract (`OUTPUT_COLUMNS`, `core/pipeline.py:19`), so `core/train.py` and `api/main.py` never change between synthetic and real data — they both import `MODEL_FEATURES`. Swapping data sources is a flag, not a refactor, which is the honest version of the "one flag away" claim. `write_track_constants()` (`:316–328`) then re-exports `data/track_constants.json` from the same `CIRCUITS` dict the labels were built with, so the serving memory and the training physics can no longer disagree about a circuit's name or its pit-lane delta.

`data/mining_run.log` is a leftover from an earlier real-mining attempt: it records FastF1 returning 0 rows and falling back. The wording in it ("High-Confidence Synthetic Fallback Engine", "250 rows") no longer matches the current code's messages.

---

11. Honest limitations (and drift found while writing this doc)

These are all read out of the actual code, not inferred from the README. Fix-worthy items are marked; items 5–8 record defects that were found while documenting this repo and **are now fixed in code**, with what remains open.

Project-level honesty

1. **The headline model is trained on simulated physics, not real telemetry.** The 2026 FastF1 live-timing archive needs an authenticated F1 account, and the 2.3.x FastF1 line breaks on the retired Ergast API (hence the `fastf1>=3.2.0` pin and the note in `requirements.txt`). The `mining_run.log` below confirms the real path returned 0 rows here. So the 0.856-position MAE measures how well the model recovers the generator's own physics. It is a genuine, correctly-validated ML result *about that data* — it is not a claim of race-grade accuracy. And since the rejoin columns are inputs to the same simulator that writes the labels, most of the 40% → 74% jump is the model being handed mechanisms it previously had to guess at; see the note in §6 before quoting that number anywhere.
2. **The Jev AI text scout is a deterministic keyword classifier**, not a language model (`api/jev_scout.py`). It is honest scaffolding: `await asyncio.sleep(0.04)` simulates the round-trip, and the async signature is the seam where a real model goes. It still contributes *displayed indicators only* — its outputs never feed the tree model, so radio text cannot change a verdict today.
3. `safety_car_escalation_risk` **is a 3-tier keyword match** (0.95 on "SAFETY CAR", 0.55 on crash/debris/incident, else 0.05) over an upper-cased concatenation, and it is *not* used in the BOX/STAY decision. This is now a sharper gap than it looks: the single most strategy-relevant fact a race control message can carry — a deployed safety car roughly **halves** the effective pit-lane delta, which is precisely when the rejoin term flips — reaches the server as a float for a progress bar and nowhere near the feature row. Fixing it means deciding whether SC state is a feature (retrain with the label contract) or a documented adjustment to `pit_loss_s`; the tree has only ever seen pit losses in 17.0–24.5 s, so feeding it a 10 s value today would be out of distribution.
4. **The traffic model is one car deep.** `rejoin_gap_s` reasons about the car immediately ahead and the one immediately behind. It does not model the queue already in the pit lane, who else boxes on the same lap, the cars you inherit behind you after the stop, or double-stack penalties. Real "traffic after the stop" is a multi-agent problem; this is a well-specified two-agent approximation of it.
5. **Unofficial project.** No team, league or broadcast integration; no tyre-model certification; portfolio/learning scope.

Fixed in code (was drift found while writing this doc)

6. **Two circuit catalogues that didn't agree → now one source.** This used to be the ugliest finding: the generator knew 20 circuits, `data/track_constants.json` had 12, only 9 overlapped, three dropdown names (`Abu Dhabi Grand Prix`, `Monza Circuit`, `Silverstone Circuit`) had never been trained on, and `core/pipeline.py` spelled Abu Dhabi `"Ab Dhabi Grand Prix"`. `write_track_constants()` (`core/pipeline.py:316-328`) now regenerates the JSON from `CIRCUITS` itself, so the names, pit losses, traffic indices and lap counts the server serves are literally the numbers the labels were built from. **Verified**: 20 tracks in the JSON, all 20 present in `CIRCUITS`, `/healthz` reports `track_count: 20`, and the GroupKFold fold printouts list `Abu Dhabi Grand Prix` correctly. *Still open:* an unknown `track_id` does not error — CatBoost routes unseen categoricals to a default leaf, so a caller who POSTs "Rutland Speedway" gets `track_fallback_used: true`, `DEFAULT_PIT_LOSS_S`, and a silently degraded prediction rather than a 4xx. Consider rejecting unknown circuits outright.
7. `traffic_index` **read then ignored, and** `pit_loss` **had an inverted sign → both resolved.** `track_constants.json` is now load-bearing: `api/main.py:99-101` reads the circuit's `pit_loss`, feeds it into `pit_loss_s` and `rejoin_gap_s` in the feature row, and echoes it back in `rejoin_context` so the operator can see which rejoin situation the model was shown. The generator's `box_cost` sign is now `+ 0.15 × (pit_loss − 20)`. **Verified consequence:** Zandvoort (18.5 s lane, traffic index 4) now carries a *worse* margin than Silverstone (21.5 s lane, index 2) at the identical snapshot — cheap lane is not automatically a cheap stop, which is the

interaction the two circuits disagree about in reality. *Remaining:* `traffic_index` itself is generator-only physics; it is not a served feature, and there is no pit-lane queue model to attach it to.

8. **`compound_degradation` was documented as "derived at inference"** while nothing derived it → the stale comment is gone and §5 states the real reason the column exists: it is generator physics, deliberately excluded from `MODEL_FEATURES` because `tire_compound` already carries it.
9. **The feature list was hand-duplicated in three places** (`OUTPUT_COLUMNS`, `train.FEATURES`, `api/main.py FEATURES`) with no shared import and no test → both consumers now `import MODEL_FEATURES as FEATURES` from `core.pipeline` (`core/train.py:13`, `api/main.py:15`). One definition, so a renamed column changes training and serving together or breaks both loudly. `model_metadata.json` still records the schema it was trained with (`features`, 11 names), which is the artefact to diff against if a future CSV and a future server disagree.

Still open

10. **`docs/screenshots/` is empty** while `.gitignore` deliberately keeps `!docs/screenshots/*.png` and the README tells you to capture `manual_verdict.png` / `live_feed.png`. The showcase images will not render until committed.
11. **Live-mode input fields are placeholders** (`api/openf1_streamer.py:161–167`): tyre age from a lap-time modulo, compound hard-coded "MEDIUM", gaps hard-coded 1.0/1.0 — OpenF1's free tier has no interval feed. So on a real Sunday the traffic-after-the-stop term does no work at all in `--live`; the replay fixture and the manual tab are where it is exercised.
12. **Generated artifacts are gitignored by design** (`models/*.cbm`, `data/historical_sc_features.csv`, the bridge, the FastF1 cache, `catboost_info/`), with `models/model_metadata.json` force-kept and `data/track_constants.json` tracked. A fresh clone is therefore *not* runnable until you execute the two build commands — and `api/main.py:31–32` raises a clear `RuntimeError` naming the missing binary if you try.

---

12. Use cases

Primary — decision support for a pit wall or strategy room. The operator supplies a race snapshot (circuit, lap, position, tyre age/compound, gaps, pace, plus radio and race-control text) and gets, in single-digit milliseconds, two finishing projections, the positional value of boxing, and the rejoin geometry the call was based on. Because the payload is plain JSON over HTTP, the same engine drops into a Slack bot, a strategy-room tablet, a broadcast graphic service, or a team dashboard without a UI change. The natural operating pattern is *replay-then-live*: rehearse against recorded fixtures, switch to `--live` when a session is actually on track, and let the auto-revert handle a blank grid.

Secondary — historical back-testing and what-if analysis. Feed a stored race frame-by-frame and log `expected_position_gain` per lap to find the moments where the engine was most confident, then score those calls against what actually happened. That's the roadmap's back-testing harness, and it's the workflow that turns a demo into a measurable claim. It also works as an interactive what-if tool: nudge `gap_ahead` in the manual tab and watch the BOX/STAY boundary cross at exactly the circuit's pit-lane delta, or sweep `tire_age` through the compound's cliff and watch the margin turn positive.

Strategy-mechanics teaching. The counterfactual contract in §7 is a clean, small, real example of causal-style reasoning with a supervised model: same snapshot, flip one decision flag, compare outcomes. Because it is circuit- and traffic-aware *by learning* rather than by a hard-coded rule (verified: identical 20-lap-old Softs → STAY OUT at `Monaco Grand Prix` by 5.74 positions, at `British Grand Prix` by only 0.96, and the British call turns at 24 laps of wear or whenever the car ahead opens a gap wider than the pit lane), it also demonstrates why you group your cross-validation by deployment unit (§6) and why you never double-count a mechanism you already trained into the model (§7). The zero-dependency SPA makes the abstractions visible — request in, verdict, rejoin geometry and latency out.

Low-latency serving as an engineering showcase. The systems story is arguably the most transferable part: two-process isolation of network I/O from inference, GIL-releasing C++ predict pushed onto a thread pool via `run_in_executor` so the async server never blocks, an atomic single-writer JSON bridge in place of a queue dependency, native compiled model serving that sidesteps ONNX categorical-export drift, `PROJECT_ROOT` path anchoring for Windows/CWD fragility, and a data-source fallback threshold so the build never breaks. Every one of those is a real production pattern, exercisable in a ~1,500-line repo (978 lines of Python, 530 of HTML/CSS/JS) with no build step and no database.

Content and fan analytics. Race-day companion: run the replay fixture behind a live stream or a fan forum thread and get an animated, explainable BOX/STAY margin per lap.

Reference architecture to reuse. Nothing here is F1-specific in shape. Any domain that must compare "act now" against "hold" under a per-context cost structure and needs sub-200 ms answers from a small tree model fits this skeleton — pit stops and fleet charging, grid dispatch vs. storage, inventory allocation vs. expedite, queue admission control. Swap `CIRCUITS` and the label physics, keep the pipeline/train/serve/bridge split.

Who benefits today, realistically: the author as a portfolio artifact, students of ML systems as a readable worked example, and motorsport-strategy enthusiasts as an explorable simulator — not as a substitute for a team's certified strategy tool.

---

13. Runbook (PowerShell, one line each)

```
pip install -r requirements.txt
python -m core.pipeline
python -m core.train
python -m uvicorn api.main:app --port 8000
```

Then open `http://127.0.0.1:8000` (dashboard) and `http://127.0.0.1:8000/docs` (Swagger). To animate the Live tab, start ingest in a second terminal:

```
python -m api.openf1_streamer
python -m api.openf1_streamer --live
python -m core.pipeline --real
```

`--real` expects `FASTF1_F1_TV_USERNAME` / `FASTF1_F1_TV_PASSWORD` to be set; without them it falls back to the generator by design. The API and the streamer must both be running for the Live tab to show frames; the manual tab works with only the API up.

Verify a running server — `GET /healthz` returns `model_loaded`, `model_version` (the training timestamp), RMSE/MAE/baseline, `total_training_rows`, `track_count`. That single call is the fastest way to confirm the shipped model, the metrics, and the serving memory all agree.

API reference

Method	Route	Purpose	Failure behaviour
POST	`/api/predict_counterfactual`	BOX vs STAY for one snapshot	422 from Pydantic range guards; 500 with a structured detail on tree-math errors; 500 "Inference cluster not initialized" if the lifespan never loaded

Method	Route	Purpose	Failure behaviour
GET	/api/live_state	Bridge snapshot + decision + rolling feed	{"available": false} on missing/locked/corrupt bridge
GET	/api/tracks	Circuit names in serving memory	empty list if constants didn't load
GET	/healthz	Readiness + provenance	—
GET	/	SPA (same origin as API → no CORS)	static mount, registered after API routes

Response shape of the main route:

```
{"status": "success",
 "jev_scout_indicators": {"safety_car_escalation_risk": 0.05, "tire_urgency_label": "Critical"},
 "counterfactual_projections": {"BOX_NOW_EXPECTED_FINISH": 5.07, "STAY_OUT_EXPECTED_FINISH": 5.54},
 "pit_wall_recommendation": "BOX",
 "expected_position_gain": 0.47,
 "rejoin_context": {"pit_loss_s": 21.5, "rejoin_gap_s": 5.9, "rejoins_behind_car_ahead": false},
 "track_fallback_used": false,
 "inference_ms": 38.46}
```

`rejoin_context` is diagnostics only — it reports the traffic situation the model was shown and is never re-scored, so the counterfactual contract (§7) stays intact.

Example request (the one above, and it is the case worth trying because it shows the clean-air rejoin winning):

```
Invoke-RestMethod -Uri http://127.0.0.1:8000/api/predict_counterfactual -Method Post -ContentType
'application/json' -Body '{"track_name":"British Grand
Prix","laps_remaining":13,"current_position":4,"tire_age":25,"tire_compound":"SOFT","gap_ahead":27.4,"gap_
behind":0.4,"driver_pace_index":90.0,"radio_transcript":"front left tyre grip is falling
away","race_control_text":"GREEN FLAG"}'
```

Drop `gap_ahead` to `1.0` with everything else unchanged and the same 25-lap-old Soft reads STAY OUT: that pair of calls is the whole traffic-after-the-stop mechanism in two requests.

---

14. What's next, in dependency order

1. ~~**Reconcile the circuit catalogues and centralise the feature list.**~~ **Done.** `track_constants.json` is generated from `CIRCUITS` by `write_track_constants()`, both consumers import one canonical `MODEL_FEATURES`, and the fold printouts confirm all 20 circuit names agree end to end.
2. ~~**Give `pit_loss` a real, documented role.**~~ **Done** — it is a served feature, it is echoed in `rejoin_context`, and the sign bug in the label physics is fixed. `traffic_index` remains generator-only until there is a pit-lane queue model to attach it to.
3. **Decide what a safety car does to the decision.** Today it changes a progress bar and nothing else. Either make SC state a feature (retrain with labels that encode the halved pit-lane delta) or apply a documented post-hoc band to `pit_loss_s` — but do not feed the tree an out-of-distribution 10 s stop without saying so.
4. **Beyond one car deep.** Multi-car rejoin: the pit-lane queue, double-stacking, and who else boxes on the same lap. That is the version of "traffic after the stop" a real strategy group runs, and `rejoin_gap_s` is the single-car approximation of it (§11 item 4).

5. **Let the text scout actually influence the decision**, once it is a real classifier: either as a model feature (retrain with the label contract) or as an explicit adjustment documented in the response. Today it is display-only.
6. **Real-data retrain** on an authenticated machine (`--real`) so the headline metrics describe actual Grand Prix telemetry — and so the cliff laps and the seconds-per-lap wear rates are fitted rather than priors.
7. **Back-test harness** over recorded races, reporting per-lap gain against realised results.
8. **WebSocket/MQTT ingest** to replace the 1.5 s bridge poll with push, plus a real interval feed so live `gap_ahead` / `gap_behind` stop being placeholders. Stint-plan optimiser and driver-vs-driver pace differentials after that.

---

15. Glossary

Term	Meaning here
Box	Pit-lane entry this lap ("box, box, box").
Stint	One tyre's run: laps between two pit stops.
Counterfactual	Same observed state, decision flag toggled; the two predictions are the two universes.
Pit loss	Seconds lost entering, servicing, and rejoining the pit lane. A per-circuit constant here (`pit_loss_s`), 17.0–24.5 s.
Rejoin gap	`gap_ahead − pit_loss_s`. Negative means the stop leaves you behind the car you were attacking; positive means the gap already covered the loss and you rejoin in clean air.
Undercut	Pitting to gain time on a car ahead while he stays out. In this model it is simply the case where `rejoin_gap_s` is positive.
Dirty air	Lap time lost following another car closely, from the reduced rear downforce behind it. Costs the driver who rejoin behind someone.
Tyre cliff	The stint length past which a compound's lap time falls away non-linearly (`COMPOUND_CLIFF_LAP`: SOFT 15, MEDIUM 22, HARD 30 laps).
Traffic index	Positions lost by joining the queue behind slower cars. Generator physics today; not a served feature.
Categorical feature	A non-numeric label (`track_id`) that CatBoost handles natively instead of one-hot encoding.
GroupKFold	Cross-validation where all rows of one group (here, one circuit) stay on the same side of the split.
RMSE / MAE	Root-mean-square / mean absolute prediction error, expressed in finishing positions.
`.cbm`	CatBoost's native compiled-model binary, loaded by C++ at serve time.
GIL	Python's Global Interpreter Lock; C++ predict releases it, which is why the thread pool genuinely parallelises.
Bridge	The shared JSON file that carries ingest state to the server without a queue or database.

Documented from source at commit `1703870` ("feat: F1 counterfactual pit-strategy engine") plus the traffic-after-the-stop change layered on top of it: the 11-feature schema and its canonical `MODEL_FEATURES` contract, the per-compound wear cliff, the rejoin physics in the generator, `track_constants.json` generated from `CIRCUITS`, and the `rejoin_context` read-out in the API and UI. Where this file and README.md disagree, §11 explains which is right and why.